Cloud Stock Watcher

mo1378al-s Mohamed Albakkour
ya5683al-s Yasin Alghabra

December 2025

1 Introduction

This project involved building a cloud-native application named Cloud Stock Watcher. The focus was on scalability, resilience, automation, and good cloud practices rather than a fully realistic trading platform.

The app lets users view real stock prices, set alert thresholds, and receive notifications when prices cross those levels. We run everything on a Kubernetes cluster set up on an IaaS provider. All components are containerised and deployed through manifests, making the setup easy to reproduce with scripts.

2 Architecture Overview

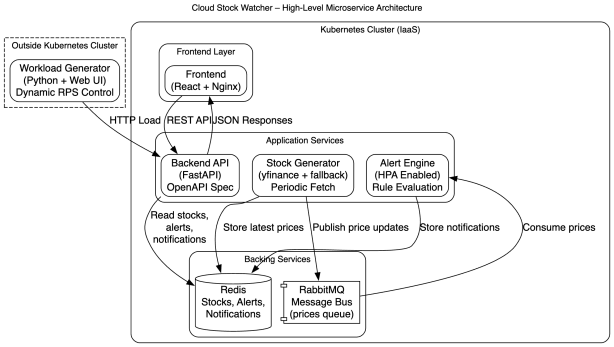

Figure 1: High-level architecture of the Cloud Stock Watcher system Using Free Graphviz(Online version)

The system is built from independent microservices. A React frontend is served by Nginx. The FastAPI backend provides REST endpoints for stock data, alerts, and notifications.

A stock generator service periodically fetches real market prices (using yfinance for symbols like TSLA, AAPL, MSFT, NVDA, and GOOG) and publishes updates to a RabbitMQ queue. It also writes the latest prices directly to Redis for quick access by the API and frontend.

An alert engine consumes messages from the queue and checks prices against user-defined rules, storing alerts and notifications in Redis. RabbitMQ decouples the generator from the engine, while Redis acts as the shared data store.

We added an external workload generator outside the cluster to create variable traffic and test autoscaling and resilience.

3 Role of RabbitMQ

RabbitMQ is a key element in the design. The stock generator publishes new prices to a queue rather than calling the alert engine directly. The alert engine then consumes these messages at its own pace. This asynchronous pattern avoids tight coupling between the services and ensures no price updates are lost if the alert engine is down or restarting.

It also enables straightforward horizontal scaling: multiple alert engine pods can consume from the same queue in parallel. Overall, this satisfies the course requirement for asynchronous communication between services and makes the system considerably more robust.

4 Stock Data Generation

We started with purely random synthetic prices to get the architecture working quickly and test message flow without external issues.

Later, we switched to real data using the yfinance library for more realism. The generator now fetches current (or recent) prices for symbols like TSLA, AAPL, MSFT, NVDA, and GOOG. If the live call fails, it falls back to the last closing price; if that's unavailable, it uses a stored value with small random variation. This keeps the service running reliably even if Yahoo Finance is down.

Each update is published as JSON to RabbitMQ and also written to Redis so the frontend can read the latest prices instantly.

The external dependency is handled defensively, and all settings (symbols, interval, etc.) come from environment variables. The core architecture stayed the same: the generator remains stateless and communicates only through RabbitMQ and Redis.

5 Deployment and Automation

Everything is managed as Infrastructure-as-Code. Kubernetes YAML manifests declare deployments, services, config maps, secrets, and autoscaling policies. The entire system can be brought up or torn down using simple scripts, with no need for manual clicks in web consoles.

We also set up a basic CI/CD pipeline that builds the Docker images and pushes them to the GitLab Container Registry whenever code changes are merged. This supports immutable deployments and zero-downtime rolling updates.

6 Autoscaling

Autoscaling is implemented via a Horizontal Pod Autoscaler targeting the alert engine. The Kubernetes metrics server collects CPU usage, and when the external workload generator ramps up traffic, the alert engine automatically scales out to additional replicas. When the load drops again, it scales back in. We verified this behaviour using standard kubectl commands during the demonstration.

7 Resilience

The services are designed to be stateless, with all persistent data kept in Redis and pending messages buffered in RabbitMQ. This means individual pods can be restarted or even terminated without affecting correctness.

Kubernetes automatically replaces failed pods, and we tested the system by deliberately killing pods while it was running. The application continued to function correctly, showing the intended level of resilience.

8 Workload Generator and Observability

The external workload generator not only creates load but also provides a simple web dashboard to adjust the request rate in real time and view basic metrics such as success rate and average latency. This meets the requirement for controllable dynamic load while keeping the overall system reasonably simple.

9 Alignment with the 12-Factor App Methodology

The project deliberately follows the principles outlined in the 12-Factor App methodology:

1. Codebase: Single Git repository, multiple deployments.
2. Dependencies: All dependencies are declared in container images.
3. Config: Configuration is externalized via environment variables, ConfigMaps, and Secrets.
4. Backing Services: Redis and RabbitMQ are treated as replaceable services.

5. Build, Release, Run: CI/CD builds immutable images; runtime config is injected at deploy time.
6. Processes: All services are stateless; state is stored in Redis.
7. Port Binding: Each service exposes a port and is routed by Kubernetes.
8. Concurrency: Services scale horizontally via Kubernetes replicas.
9. Disposability: Pods can start/stop at any time without affecting correctness.
10. Dev/Prod Parity: Same containers and manifests are used across environments.
11. Logs: Services log to stdout/stderr and are managed by Kubernetes.
12. Admin Processes: Administrative actions are performed using the same container environment.

10 Conclusion

The project meets all the pass-level requirements of the Cloud Computing course. It presents a solid, well-structured cloud-native architecture that makes effective use of Kubernetes. It includes asynchronous message-based communication, demonstrates autoscaling and resilience under load, and is fully automated and reproducible.

All design decisions were made thoughtfully and in line with established cloud best practices. The result is a complete and demonstrable system that should provide a clear foundation for a passing grade.

Criterion	**Status**	**Motivation**
Project description written before implementation, including OpenAPI specification and high-level architecture diagram	Fulfill	A project plan and high-level architecture description were written before implementation, including an OpenAPI specification and an architecture diagram.
Application runs on a self-deployed Kubernetes cluster on an Infrastructure-as-a-Service platform	Fulfill	The application is deployed on a self-managed Kubernetes (k3s) cluster running on Infrastructure-as-a-Service virtual machines.
Application consists of at least three interacting microservices	Fulfill	The system consists of multiple interacting microservices, including Frontend, Backend API, Stock Generator, and Alert Engine.

Criterion	Status	Motivation
At least one microservice uses a message bus (e.g., RabbitMQ or Redis Streams)	Fulfill	RabbitMQ is used as a message bus to asynchronously transmit stock price updates from the Stock Generator to the Alert Engine.
One microservice stores data for the lifetime of the application	Fulfill	Redis is used as a backing service to store stock prices, alert rules, and notifications for the lifetime of the application.
Application has a graphical frontend (webpage)	Fulfill	A React-based frontend served via Nginx provides a graphical user interface for interacting with the system.
Application exposes an API according to an OpenAPI specification	Fulfill	The Backend API exposes REST endpoints that are formally documented using an OpenAPI specification.
External workload generator (outside Kubernetes cluster)	Fulfill	A Python-based workload generator runs outside the Kubernetes cluster and generates HTTP traffic toward the Backend API.
Workload generator allows dynamic workload changes via API, GUI, or similar	Fulfill	The workload generator provides an HTTP API and web interface that allow runtime adjustment of request rate and workload behavior.
All configuration provided via ConfigMaps and Secrets	Fulfill	Configuration such as service endpoints, credentials, and update intervals is externalized using Kubernetes ConfigMaps and Secrets.
Autoscaling demonstrated in response to workload changes	Fulfill	Kubernetes Horizontal Pod Autoscalers are configured and demonstrated for selected services under increased workload.
Resilience demonstrated when one or more pods are killed	Fulfill	Kubernetes automatically restarts terminated pods while the application remains functional.
Resilience demonstrated when a worker node becomes non-functional	Fulfill	Kubernetes reschedules pods onto available nodes when a worker node becomes unavailable.

Criterion	Status	Motivation
Rolling update demonstrated using a CI/CD pipeline	Fulfill	A GitLab CI/CD pipeline builds and pushes container images, enabling rolling updates in Kubernetes.
Application can be brought up and torn down using scripts only	Fulfill	Deployment and teardown of the application are fully script-based using Kubernetes manifests and shell scripts.
Short written report (1–2 pages) explaining design decisions and usage	Fulfill	A concise written report extending the README explains architectural decisions and usage instructions.
Report addresses all criteria and explains why the project should pass	Fulfill	The report explicitly maps implementation details to all pass-level project requirements.
12-Factor App principles referenced and explained	Fulfill	The report explains how the application follows the 12-Factor App principles.

Table 1: Pass-level requirement checklist for the Cloud Stock Watcher project